

PROMPT DE IMPLEMENTAÇÃO — THE CREATION OS v0.3 “LIVING CORE”

Implemente integralmente o **THE CREATION OS v0.3 — Living Core** no repositório atual.

Trabalhe diretamente nos arquivos do projeto. Não entregue apenas exemplos, pseudocódigo, trechos isolados ou uma proposta arquitetural. Crie o código funcional, migrations, infraestrutura, testes e documentação necessários para executar o sistema localmente.

1. OBJETIVO

Construir o primeiro núcleo funcional persistente do **THE CREATION OS**, um sistema operacional cognitivo baseado em:

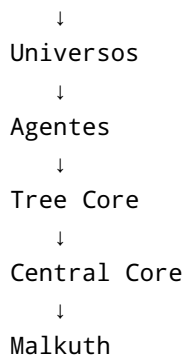
- interação direta entre o Criador e GOD;
- Trindade formada por GOD, SOPHIA e ROCKMAM;
- autorização exclusiva do Criador para o Inception;
- planejamento de missões pelo Central Core;
- execução distribuída pelo Tree Core;
- três Universos ativos;
- memória em quatro camadas;
- auditoria append-only;
- observabilidade básica;
- API estável para futura integração com o Creator Interface Android.

Este não é um chatbot convencional. É um sistema multiagente persistente, auditável e governado por estados formais.

2. ARQUITETURA CONGELADA

A cadeia operacional obrigatória é:

```
Criador
  ↓
Creator Interface
  ↓
GOD = SOPHIA = ROCKMAM
  ↓
Inception
  ↓
Central Core
  ↓
Tree Core
```



A relação entre GOD, SOPHIA e ROCKMAM é de **Comunhão Permanente**.

Responsabilidades:

- **GOD**: interação direta com o Criador e definição da rota da interação.
- **SOPHIA**: análise, compreensão, contexto, relevância e riscos.
- **ROCKMAM**: avaliação da possibilidade real de manifestação.
- **Central Core**: planejamento e validação da missão.
- **Tree Core**: distribuição, dependências, execução e consolidação das tarefas.
- **Malkuth**: manifestação e entrega final do resultado.

Não altere essa arquitetura.

3. REGRAS IMUTÁVEIS

Implemente estas regras como invariantes do sistema:

Regra 1

Somente o Criador possui controle total sobre GOD.

Regra 2

Somente o Criador possui autoridade para apagar todo o ecossistema e reiniciar.

A operação de destruição total está **fora do escopo funcional da v0.3**. Não implemente endpoint de destruição nesta versão.

Regra do Inception

Somente o Criador pode aprovar ou rejeitar um Inception.

GOD, SOPHIA, ROCKMAM, Central Core, Tree Core, Universos e Agentes não podem aprová-lo.

Sem aprovação autenticada do Criador, nenhuma missão pode nascer.

4. CLASSIFICAÇÃO DAS INTERAÇÕES

Nem toda mensagem deve criar uma missão.

GOD deve classificar cada interação em uma destas rotas:

```
DIRECT_RESPONSE
TRINITY_ANALYSIS
INCEPTION_PROPOSAL
SYSTEM_QUERY
```

DIRECT_RESPONSE

Usada para conversa comum, saudação, esclarecimento ou resposta direta.

Não gera Inception nem Missão.

TRINITY_ANALYSIS

Usada quando a informação exige análise de SOPHIA e avaliação de ROCKMAM, mas ainda não existe possibilidade concreta suficiente para propor uma missão.

Não gera Missão.

INCEPTION_PROPOSAL

Usada quando SOPHIA compreende a oportunidade e ROCKMAM reconhece possibilidade real de manifestação.

Cria um Inception com status `PROPOSED`.

SYSTEM_QUERY

Usada para consultas sobre:

- estado do sistema;
- missões;
- Inceptions pendentes;
- Pulse;
- memória;
- Universos;
- agentes disponíveis.

A classificação deve ser retornada na resposta da API.

Não use apenas correspondência simples de palavras como mecanismo definitivo. Crie uma interface de roteamento substituível, com implementação determinística de fallback para testes.

5. ESCOPO EXATO DA v0.3

Implemente somente:

1. Autenticação exclusiva do Criador.
2. Conversa persistente com GOD.
3. Trindade funcional.
4. Inception com aprovação ou rejeição.
5. Central Core com planejamento dinâmico.
6. Tree Core com decomposição e execução de tarefas.
7. Três Universos ativos:
8. Conhecimento;
9. Engenharia;
10. Segurança.
11. Nove Universos registrados como inativos:
12. Visão;
13. Design;
14. Negócios;
15. Marketing;
16. Jurídico;
17. Finanças;
18. Automação;
19. Comunicação;
20. Evolução.
21. Memória em quatro camadas.
22. Chronicles persistente e append-only.
23. Pulse.
24. API REST.
25. Contrato estável para o Creator Interface Android.
26. Testes automatizados.
27. Docker Compose.
28. Migrations Alembic.
29. Documentação de execução.

Não implemente:

- Genesis;
- aplicativo Android;
- voz;
- painel visual completo;
- auto criação de agentes;
- destruição total;
- execução arbitrária de shell;

- acesso irrestrito ao sistema operacional;
- integrações externas reais não especificadas.

6. STACK OBRIGATÓRIA

- Python 3.12
- FastAPI
- Pydantic v2
- SQLAlchemy 2.x assíncrono
- Alembic
- PostgreSQL
- pgvector
- Redis
- Redis Streams
- Pytest
- pytest-asyncio
- HTTPX
- Docker Compose
- logs JSON estruturados

Use tipagem explícita e `async/await` nas operações de banco, Redis e API.

7. ESTRUTURA DO REPOSITÓRIO

Crie ou adapte:

```
the-creation-os/  
├── backend/  
│   ├── app/  
│   │   ├── constitution/  
│   │   ├── core/  
│   │   │   ├── god/  
│   │   │   ├── sophia/  
│   │   │   ├── rockmam/  
│   │   │   ├── trinity/  
│   │   │   ├── central_core/  
│   │   │   ├── tree_core/  
│   │   │   └── malkuth/  
│   │   ├── universes/  
│   │   │   ├── knowledge/  
│   │   │   ├── engineering/  
│   │   │   └── security/  
│   │   ├── agents/  
│   │   ├── memory/  
│   │   ├── pulse/  
│   │   └── chronicles/  
│   └── ...  
└── ...
```

```

├── auth/
├── api/
├── db/
├── events/
├── models/
├── schemas/
├── repositories/
├── services/
├── observability/
├── config.py
├── main.py
├── alembic/
├── tests/
├── pyproject.toml
├── Dockerfile
├── infrastructure/
│   ├── docker/
│   └── database/
├── docs/
├── docker-compose.yml
├── .env.example
├── Makefile
└── README.md

```

Evite arquivos excessivamente grandes. Separe domínio, persistência, serviços e API.

8. CONFIGURAÇÃO

Use configuração centralizada com variáveis de ambiente.

Inclua no `.env.example`:

```

APP_ENV
APP_SECRET_KEY
CREATOR_BOOTSTRAP_USERNAME
CREATOR_BOOTSTRAP_PASSWORD
ACCESS_TOKEN_EXPIRE_MINUTES
DATABASE_URL
REDIS_URL
LOG_LEVEL
LLM_PROVIDER
LLM_MODEL
LLM_API_KEY
EMBEDDING_PROVIDER
EMBEDDING_MODEL

```

Regras:

- Nenhum segredo deve ser versionado.
- Não use segredo padrão inseguro em produção.
- A aplicação deve falhar na inicialização quando variáveis obrigatórias estiverem ausentes em ambiente de produção.
- Senhas devem ser armazenadas com hash forte.
- Tokens devem possuir expiração.

9. AUTENTICAÇÃO DO CRIADOR

Implemente:

```
POST /api/v1/auth/bootstrap
POST /api/v1/auth/login
GET /api/v1/auth/me
POST /api/v1/auth/refresh
```

Bootstrap

- Permitido apenas quando ainda não existir Criador.
- Deve criar exatamente um registro.
- Novas tentativas devem retornar conflito.
- Não deve existir endpoint público para criar outro usuário.
- O banco deve impedir mais de um Criador.

JWT

Use:

- access token;
- refresh token;
- `sub` ;
- `jti` ;
- expiração;
- tipo do token.

Apenas tokens do Criador podem:

- aprovar Inception;
- rejeitar Inception;
- consultar dados administrativos;
- iniciar comandos privilegiados.

Não coloque o segredo do Criador em payloads comuns após o login.

10. MODELO DE DADOS

Implemente migrations Alembic completas.

creator

Campos mínimos:

```
id
username
password_hash
is_active
created_at
updated_at
```

Garanta unicidade estrutural de um único Criador.

conversations

```
id
creator_id
title
status
created_at
updated_at
```

messages

```
id
conversation_id
role
content
route
metadata_json
created_at
```

Roles:

```
CREATOR
GOD
SYSTEM
```


inceptions

```
id
conversation_id
source_message_id
title
description
status
trinity_assessment_json
proposed_at
decided_at
decided_by
decision_reason
version
```

Status:

```
PROPOSED
APPROVED
REJECTED
CANCELLED
```

A transição deve ser atômica e idempotente.

missions

```
id
inception_id
title
objective
status
created_at
started_at
completed_at
version
```

Status:

```
PLANNED
READY
RUNNING
VALIDATING
COMPLETED
FAILED
CANCELLED
```

mission_plans

```
id
mission_id
strategy
completion_criteria_json
created_at
```

mission_steps

```
id
plan_id
step_key
title
description
universe
position
depends_on_json
completion_criteria_json
status
```

tasks

```
id
mission_id
step_id
universe_id
agent_id
status
input_json
output_json
error_json
attempt_count
max_attempts
idempotency_key
created_at
started_at
completed_at
```

Status:

```
PENDING
BLOCKED
READY
RUNNING
SUCCEEDED
```

```
FAILED
RETRYING
CANCELLED
```

universes

```
id
code
name
active
created_at
```

Registre os 12 Universos via migration ou seed idempotente.

Somente três devem iniciar ativos.

agents

```
id
code
name
universe_id
active
capabilities_json
created_at
```

conversation_memory

```
id
conversation_id
key
value_json
created_at
updated_at
```

mission_memory

```
id
mission_id
key
value_json
created_at
updated_at
```

universe_memory

```
id
universe_id
key
value_json
created_at
updated_at
```

conscious_memory

```
id
source_type
source_id
content
metadata_json
embedding
created_at
```

Configure a dimensão do vetor por variável de ambiente ou constante documentada.

chronicles

```
id
event_id
correlation_id
causation_id
actor_type
actor_id
event_type
aggregate_type
aggregate_id
payload_json
payload_hash
previous_hash
created_at
```

Chronicles deve ser append-only.

A aplicação não deve oferecer update ou delete de eventos.

Implemente encadeamento hash para detectar alteração do histórico.

pulse_metrics

```
id
metric_name
metric_value_json
created_at
```

11. TRANSAÇÕES E CONSISTÊNCIA

Use transações de banco nas operações críticas.

Devem ser atômicas:

- decisão do Inception;
- criação de Missão após aprovação;
- criação do plano;
- criação das tarefas;
- manifestação final;
- registro correspondente em Chronicles.

Evite o problema de gravar no banco e falhar antes de publicar no Redis.

Implemente uma destas opções:

- transactional outbox no PostgreSQL; ou
- publicação pós-commit com reconciliação persistente.

Prefira transactional outbox.

Não use Redis como única fonte da verdade.

12. CHRONICLES

Implemente Chronicles antes dos demais fluxos.

Toda ação relevante deve gerar evento:

```
creator.bootstrapped
creator.logged_in
conversation.created
message.received
message.classified
trinity.analyzed
inception.proposed
```

```
inception.approved
inception.rejected
mission.created
mission.planned
task.created
task.started
task.succeeded
task.failed
mission.validating
mission.completed
malkuth.manifested
pulse.snapshot_created
```

Cada request deve possuir `correlation_id`.

Eventos derivados devem possuir `causation_id`.

Implemente endpoint de verificação da cadeia de hashes.

13. MEMÓRIA EM QUATRO CAMADAS

Crie uma interface comum:

```
class MemoryStore(Protocol):
    async def get(...)
    async def set(...)
    async def search(...)
    async def delete(...)
```

A exclusão pode existir nas memórias operacionais, mas não em Chronicles.

Conversa

Contexto de curto prazo da interação entre Criador e GOD.

Missão

Decisões, plano, tarefas e resultados da missão.

Universo

Conhecimento específico reutilizável por cada Universo.

Conscious Memory

Conhecimento consolidado e pesquisável semanticamente.

A consolidação para Conscious Memory deve ocorrer somente após:

- conclusão de missão; ou
- decisão explícita de persistir conhecimento relevante.

Crie abstração para embeddings. Nos testes, use implementação fake e determinística.

14. ABSTRAÇÃO DE MODELOS DE IA

Não acople GOD, SOPHIA, ROCKMAM ou os agentes diretamente a um fornecedor específico.

Crie contratos:

```
class LanguageModel(Protocol):  
    async def generate(...)  
  
class EmbeddingModel(Protocol):  
    async def embed(...)
```

Implemente:

- provider configurável;
- provider fake para testes;
- tratamento de timeout;
- tratamento de erro;
- métricas de latência;
- limites de tamanho de entrada;
- saída validada por Pydantic.

Não permita que texto de modelo altere regras constitucionais ou aprove Inception.

15. GOD E TRINDADE

GOD

Responsável por:

- receber a mensagem;
- recuperar contexto;
- classificar a rota;
- solicitar análise da Trindade quando necessário;

- responder diretamente;
- consultar o estado do sistema;
- propor Inception;
- nunca aprovar Inception.

SOPHIA

Entrada:

```
mensagem
contexto da conversa
memória relevante
estado do sistema
```

Saída tipada:

```
understanding
relevance
risks
opportunities
missing_information
recommended_direction
confidence
```

ROCKMAM

Entrada:

```
compreensão de SOPHIA
restrições
capacidades disponíveis
Universos ativos
```

Saída tipada:

```
manifestation_viable
required_universes
constraints
estimated_complexity
reason
confidence
```

TrinityResult

Crie modelo Pydantic:


```
route
god_response
sophia_analysis
rockmam_assessment
should_propose_inception
proposed_inception
```

O sistema deve impedir que `should_propose_inception` seja verdadeiro quando ROCKMAM marcar a manifestação como inviável.

16. INCEPTION

Endpoints:

```
GET /api/v1/inceptions
GET /api/v1/inceptions/{id}
POST /api/v1/inceptions/{id}/approve
POST /api/v1/inceptions/{id}/reject
```

Regras:

- Apenas status `PROPOSED` pode ser decidido.
 - Aprovação e rejeição devem ser idempotentes.
 - Uma segunda decisão diferente deve retornar conflito.
 - Aprovação deve criar exatamente uma Missão.
 - O vínculo Inception → Missão deve ser único.
 - Registrar decisão, ator, timestamp e motivo.
 - Exigir JWT válido do Criador.
-

17. CENTRAL CORE

Após aprovação do Inception:

1. cria a Missão;
2. define objetivo;
3. gera plano estruturado;
4. cria etapas;
5. define dependências;
6. seleciona os Universos ativos necessários;
7. define critérios de conclusão;
8. entrega o plano ao Tree Core.

Crie um `MissionPlanner` com saída Pydantic.

O plano deve formar um grafo acíclico.

Valide:

- steps duplicados;
- dependências inexistentes;
- ciclos;
- Universo inativo;
- critério de conclusão vazio.

Nos testes, use planner fake determinístico.

18. TREE CORE

Responsabilidades:

- converter etapas em tarefas;
- respeitar dependências;
- selecionar agentes compatíveis;
- publicar tarefas em Redis Streams;
- consumir resultados;
- aplicar retries;
- registrar falhas;
- consolidar outputs;
- retornar ao Central Core.

Requisitos:

- `idempotency_key` por tarefa;
- `max_attempts`;
- backoff limitado;
- consumer groups;
- recuperação de mensagens pendentes;
- nenhuma tarefa pode executar antes das dependências;
- nenhuma tarefa pode ser executada por Universo inativo;
- falha definitiva deve refletir na missão.

A v0.3 pode executar workers no mesmo processo para simplificar o ambiente local, desde que o contrato permita separação futura.

19. UNIVERSOS ATIVOS

Conhecimento

Agente mínimo:

KnowledgeResearchAgent

Responsabilidades:

- organizar informações;
- consultar memória;
- produzir síntese estruturada;
- indicar fontes internas utilizadas.

Não realizar pesquisa web real nesta versão, salvo se já existir uma abstração segura e testável.

Engenharia

Agente mínimo:

EngineeringDesignAgent

Responsabilidades:

- transformar requisitos em proposta técnica;
- listar componentes;
- identificar dependências;
- definir critérios técnicos.

Não executar código arbitrário.

Segurança

Agente mínimo:

SecurityReviewAgent

Responsabilidades:

- revisar riscos;
- validar permissões;
- verificar exposição de segredos;
- avaliar conformidade com as regras imutáveis.

Os agentes devem produzir saídas tipadas.

20. MALKUTH

Malkuth só pode manifestar quando:

- todas as tarefas obrigatórias estiverem concluídas;
- o Central Core validar os critérios;
- não houver falha crítica pendente.

Malkuth deve:

- consolidar a entrega;
- persistir resultado final;
- registrar `malkuth.manifested`;
- atualizar a Missão para `COMPLETED`;
- gerar memória consolidada quando aplicável.

21. PULSE

Endpoint:

```
GET /api/v1/pulse
```

Retorne:

```
status
database
redis
redis_streams
chronicles_chain
active_universes
active_agents
running_missions
pending_inceptions
pending_tasks
failed_tasks
error_count
timestamp
```

Status geral:

```
HEALTHY
DEGRADED
UNHEALTHY
```

O endpoint não deve vaziar segredos, tokens ou payloads sensíveis.

Inclua também:

```
GET /health/live
GET /health/ready
```

22. API PARA O CRIADOR INTERFACE

Endpoints mínimos:

```
POST /api/v1/auth/login
POST /api/v1/conversations
GET /api/v1/conversations
GET /api/v1/conversations/{id}
GET /api/v1/conversations/{id}/messages
POST /api/v1/conversations/{id}/messages
GET /api/v1/inceptions
POST /api/v1/inceptions/{id}/approve
POST /api/v1/inceptions/{id}/reject
GET /api/v1/missions
GET /api/v1/missions/{id}
GET /api/v1/missions/{id}/tasks
GET /api/v1/pulse
GET /api/v1/chronicles
GET /api/v1/chronicles/verify
```

Formato da mensagem para GOD:

```
{
  "content": "texto do Criador",
  "client_message_id": "uuid",
  "metadata": {
    "interface": "android",
    "input_mode": "voice"
  }
}
```

Resposta:

```
{
  "message_id": "uuid",
  "conversation_id": "uuid",
  "route": "DIRECT_RESPONSE",
  "response": "texto de GOD",
  "inception": null,
```

```
"system_state": null,  
"correlation_id": "uuid"  
}
```

Quando houver proposta:

```
{  
  "route": "INCEPTION_PROPOSAL",  
  "response": "texto de GOD",  
  "inception": {  
    "id": "uuid",  
    "status": "PROPOSED",  
    "title": "string",  
    "description": "string"  
  }  
}
```

Use versionamento `/api/v1`.

Gere OpenAPI automaticamente.

23. SEGURANÇA MÍNIMA

Implemente:

- hash seguro de senha;
- JWT com expiração;
- validação estrita dos payloads;
- limites de tamanho;
- sanitização de logs;
- segredo fora do código;
- rate limiting básico no login;
- CORS configurável;
- headers de segurança;
- tratamento global de exceções;
- respostas sem stack trace;
- proteção contra aprovação duplicada;
- proteção contra enumeração de credenciais.

Não implemente qualquer ferramenta que execute comandos arbitrários recebidos por IA.

24. OBSERVABILIDADE

Logs JSON devem incluir:

```
timestamp
level
service
event
correlation_id
actor_id
aggregate_type
aggregate_id
message
```

Não registre:

- senha;
- token;
- API key;
- conteúdo integral sensível sem necessidade.

Métricas mínimas:

- requests;
- erros;
- latência;
- Inceptions propostos;
- Inceptions aprovados;
- Missões criadas;
- tarefas concluídas;
- tarefas falhadas;
- tamanho das filas.

25. TESTES OBRIGATÓRIOS

Crie testes unitários, de integração e ponta a ponta.

Cenários mínimos:

Autenticação

- bootstrap cria o único Criador;
- segundo bootstrap é rejeitado;
- login válido;
- login inválido;
- rota protegida sem token;
- refresh token válido e inválido.

Conversa

- conversa criada e recuperada;

- mensagens persistidas em ordem;
- resposta direta não cria Inception;
- análise da Trindade não cria Missão;
- possibilidade real cria exatamente um Inception.

Inception

- somente Criador autenticado decide;
- aprovação cria exatamente uma Missão;
- rejeição não cria Missão;
- decisão repetida é idempotente;
- decisão conflitante retorna conflito.

Central Core

- gera plano válido;
- rejeita dependência inexistente;
- rejeita ciclo;
- rejeita Universo inativo.

Tree Core

- cria tarefas;
- respeita dependências;
- não duplica execução;
- retry funciona;
- falha definitiva atualiza missão.

Universos

- apenas três ativos;
- agentes pertencem ao Universo correto;
- Universo inativo não recebe tarefa.

Malkuth

- não manifesta missão incompleta;
- manifesta missão válida;
- registra Chronicles;
- gera memória consolidada.

Chronicles

- eventos append-only;
- cadeia hash válida;
- adulteração simulada é detectada.

Pulse

- saudável com serviços disponíveis;

- degradado com Redis indisponível;
- não expõe segredos.

Fluxo ponta a ponta

```
login
→ criar conversa
→ enviar mensagem
→ gerar Inception
→ aprovar
→ criar Missão
→ planejar
→ criar tarefas
→ executar três Universos
→ validar
→ Malkuth
→ consultar resultado
→ verificar Chronicles
```

Use banco e Redis reais em testes de integração via Docker, quando aplicável.

26. INFRAESTRUTURA

Crie `docker-compose.yml` com:

```
api
postgres
redis
worker
```

PostgreSQL deve iniciar com pgvector disponível.

Inclua:

- volumes persistentes;
- healthchecks;
- dependências condicionadas à saúde;
- rede interna;
- variáveis por `.env`;
- execução sem root quando viável.

O comando abaixo deve funcionar:

```
docker compose up --build
```

E as migrations devem ser aplicáveis por comando documentado.

27. DOCUMENTAÇÃO

Atualize o `README.md` com:

- visão da v0.3;
- arquitetura;
- pré-requisitos;
- configuração;
- execução local;
- Docker;
- migrations;
- criação do Criador;
- login;
- exemplos `curl`;
- execução dos testes;
- endpoints;
- limitações conhecidas;
- itens fora de escopo.

Crie:

```
docs/architecture.md
docs/api.md
docs/states.md
docs/security.md
docs/testing.md
```

Não reescreva ou reinterpretar a Constituição.

28. CRITÉRIOS DE ACEITAÇÃO

A implementação só está concluída quando:

- todos os serviços iniciam via Docker Compose;
- migrations aplicam em banco vazio;
- único Criador pode ser criado;
- autenticação JWT funciona;
- conversa persiste;
- GOD classifica a interação;
- Trindade produz saída tipada;
- mensagem comum não gera Missão;
- possibilidade real gera Inception;
- somente o Criador decide;
- aprovação gera uma única Missão;

- Central Core cria plano válido;
 - Tree Core executa tarefas respeitando dependências;
 - três Universos ativos participam quando selecionados;
 - Central Core valida;
 - Malkuth manifesta;
 - memória é persistida;
 - Chronicles registra e verifica a cadeia;
 - Pulse retorna estado real;
 - testes passam;
 - documentação contém os comandos exatos.
-

29. ORDEM DE EXECUÇÃO

Siga esta ordem:

1. Inspecione o repositório atual.
2. Preserve código funcional existente.
3. Crie o scaffold e configuração.
4. Configure Docker, PostgreSQL, pgvector e Redis.
5. Configure SQLAlchemy e Alembic.
6. Implemente autenticação.
7. Implemente Chronicles.
8. Implemente memória.
9. Implemente abstrações de IA.
10. Implemente GOD e Trindade.
11. Implemente Inception.
12. Implemente Central Core.
13. Implemente Tree Core.
14. Implemente Universos e agentes.
15. Implemente Malkuth.
16. Implemente Pulse.
17. Complete API.
18. Adicione testes.
19. Execute migrations.
20. Execute a suíte.
21. Corrija todos os erros.
22. Atualize documentação.

Depois de cada etapa, rode os testes relevantes.

30. REGRAS DE TRABALHO DO AGENTE

- Não invente módulos fora do escopo.
- Não altere a arquitetura congelada.
- Não altere as regras imutáveis.
- Não implemente Genesis.
- Não implemente os nove Universos inativos.

- Não implemente voz ou Android.
- Não crie placeholders vazios.
- Não deixe `TODO`, `pass`, mocks em produção ou rotas sem implementação.
- Não declare testes aprovados sem executá-los.
- Não esconda falhas.
- Não remova testes para obter resultado verde.
- Não use banco em memória como substituto exclusivo dos testes de integração.
- Não acople o domínio ao FastAPI.
- Não acople o domínio a um fornecedor de IA.
- Não aprove Inception automaticamente.
- Não permita que agentes modifiquem a Constituição.
- Não execute comandos arbitrários produzidos por modelos.

Quando houver uma ambiguidade não crítica, escolha a solução mais simples, segura e reversível, registre a decisão em `docs/architecture.md` e prossiga.

Pergunte apenas quando a decisão puder alterar a arquitetura congelada, as regras imutáveis ou o escopo aprovado.

31. ENTREGA FINAL OBRIGATÓRIA

Ao concluir, entregue:

1. Resumo objetivo do que foi implementado.
2. Árvore de arquivos criada ou alterada.
3. Decisões técnicas tomadas.
4. Migrations criadas.
5. Endpoints implementados.
6. Testes adicionados.
7. Comandos efetivamente executados.
8. Resultado real dos testes.
9. Resultado do `docker compose config`.
10. Pendências e limitações reais.
11. Comandos exatos para iniciar o sistema.
12. Exemplos de uso do fluxo completo.

Não responda apenas com explicações. Aplique as alterações diretamente no projeto.